

Enhancing Retrieval-Augmented Generation with Instruction-Aware Context Compression

UNIST CSE40201 Default Project

Asanali Beksultanov

Department of Computer Science
Ulsan National Institute of Science and Technology
as.beks@unist.ac.kr

Abstract

This project implements and evaluates a Retrieval-Augmented Generation (RAG) system using a custom GPT-style language model for open-domain question answering. We first construct a transformer-based decoder model (GPT-small) equipped with rotary positional embeddings (RoPE), RMSNorm, and grouped query attention (GQA), pretraining it on a large corpus and fine-tuning it on summarization and classification tasks. For the RAG pipeline, we integrate a BM25-based retriever with both our custom model and a large instruction-tuned language model (Llama-3.2-1B-Instruct), enabling dynamic retrieval and augmentation of external knowledge for each query. We explore advanced prompting strategies, including few-shot examples and chain-of-thought reasoning, and investigate context compression techniques to improve efficiency and relevance. Our results demonstrate the effectiveness of RAG for leveraging up-to-date information, highlight the impact of model size and prompting on performance, and provide insights into the challenges and opportunities for further enhancements in retrieval-augmented language models.

1 Introduction

Large language models (LLMs) like GPT have revolutionized natural language processing (NLP) by demonstrating remarkable capabilities in text generation, summarization, and question answering. However, their reliance on static training data imposes critical limitations: they cannot access new information post-training, struggle with domain-specific or time-sensitive queries, and risk generating plausible but incorrect answers (hallucinations) when faced with unfamiliar topics. These shortcomings are particularly problematic for applications requiring factual accuracy, such as medical advice, legal analysis, or real-time technical support. Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), addresses these limitations by integrating dynamic retrieval from external knowledge sources with LLM generation. In a RAG pipeline, a retriever fetches relevant documents or passages for a given query, which are then provided as context to the LLM to generate informed, up-to-date answers. This paradigm shift enables models to bypass the constraints of fixed training data while maintaining the flexibility and fluency of modern LLMs.

Despite its promise, implementing effective RAG systems presents significant challenges. Retrieval quality heavily influences answer accuracy, as irrelevant or noisy context can mislead the model. Additionally, LLMs have finite context windows (e.g., 2048 tokens for GPT-small), requiring careful management of retrieved content to avoid truncation or information overload. Recent advances, such as instruction-aware contextual compression and chain-of-thought (CoT) prompting, aim to mitigate these issues by dynamically filtering context and guiding the model's reasoning process.

In this project, we design and evaluate a modular RAG system built around a custom GPT-style transformer model. Our contributions include: Architectural Innovations: Implementing a GPT-small model with rotary positional embeddings (RoPE), RMSNorm, and grouped query attention (GQA) for

efficient training and inference. Task-Specific Fine-Tuning: Adapting the model for summarization (CNN/DailyMail) and classification (20 Newsgroups) tasks to benchmark its capabilities. RAG Pipeline Development: Integrating BM25 retrieval with advanced prompting strategies (e.g., Llama-3.2 chat formatting, CoT reasoning) and context compression techniques to optimize answer quality and computational efficiency.

Comprehensive Evaluation: Testing on the Natural Questions (NQ) dataset to quantify the impact of retrieval, compression, and prompting on metrics like accuracy and ROUGE. Our work bridges foundational research (e.g., LLaMA’s architecture, GPT-3’s few-shot learning) with practical system design, offering insights into the trade-offs between model scale, retrieval quality, and prompt engineering. By open-sourcing our implementation, we aim to provide a reproducible framework for further exploration of retrieval-augmented systems in resource-constrained environments.

2 Related Work

Recent advances in large language models (LLMs) have been driven by foundational works such as the Transformer architecture by Vaswani et al.[3], which introduced self-attention mechanisms enabling efficient sequence modeling. The development of GPT-3 by Brown et al. [2] demonstrated that large-scale autoregressive models can perform few-shot learning across a wide range of tasks. Following this, Touvron et al.[1] introduced LLaMA, a family of open and efficient foundation models that use rotary embeddings and other architectural improvements similar to our implementation. To address the limitations of static model knowledge, Lewis et al. proposed Retrieval-Augmented Generation (RAG), combining retrieval from external sources with generative models to enable up-to-date answers. More recently, Hou et al. introduced instruction-aware contextual compression, improving the efficiency and accuracy of RAG by dynamically filtering and summarizing retrieved context. Our work builds on these advancements, integrating efficient model architectures with advanced retrieval and context compression techniques.

3 Approach

Our project addresses the challenge of building a custom, small-scale GPT-style language model and integrating it into a retrieval-augmented generation (RAG) pipeline for open-domain question answering. The approach is modular and can be divided into two main phases: (1) the construction and fine-tuning of a GPT-small model, and (2) the implementation and enhancement of a RAG system using both our custom model and a large instruction-tuned model (Llama-3.2-1B-Instruct). The architecture of our GPT-small model incorporates several modern transformer improvements, including rotary positional embeddings (RoPE), RMSNorm layer normalization, and grouped query attention (GQA), which together enable efficient training and inference while maintaining strong performance on language understanding and generation tasks. After pretraining the model from scratch on a large corpus, we fine-tune it on summarization and classification tasks to adapt it to specific downstream applications.

For the RAG pipeline, we begin by retrieving relevant passages for each user query using a BM25 retriever and the Pyserini toolkit, which provides efficient and reproducible information retrieval. The retrieved passages are combined with the user’s question in a carefully constructed prompt, which is then fed into either our custom GPT-small model or the Llama-3.2-1B-Instruct model for answer generation. To further improve the quality and efficiency of the system, we experiment with advanced prompting techniques such as few-shot examples and chain-of-thought (CoT) reasoning, as well as context compression methods that filter or summarize the retrieved information to ensure only the most relevant context is used. These enhancements are designed to maximize the model’s ability to leverage external knowledge while minimizing computational overhead and the risk of information overload.

Finally, we explore additional extensions inspired by recent research, including parallel context windows (PCW) for handling long documents, and reflective retrieval methods such as Self-RAG and RAFT for domain-specific adaptation. Our implementation is highly modular, allowing for easy integration of new prompting strategies, retrieval methods, and context compression techniques. By combining these elements, our approach provides a robust and flexible framework for open-domain question answering, capable of leveraging both custom and off-the-shelf language models while

addressing the limitations of static model knowledge through dynamic retrieval and context-aware generation

4 Experiments

We evaluate our approach primarily on the Natural Questions (NQ) dataset, a widely recognized benchmark for open-domain question answering. Each query in NQ is paired with a relevant Wikipedia passage and annotated for both long and short answers; for our experiments, we focus on the short-answer subset, which better aligns with the requirements of our retrieval-augmented generation (RAG) pipeline. The input to our system consists of a user’s query and a set of up to three relevant passages retrieved from the Wikipedia corpus, formatted in a clear and structured manner to ensure compatibility with both our custom GPT-small model and the larger Llama-3.2-1B-Instruct model. Each passage is accompanied by its title and a compressed, instruction-aware summary when context compression is enabled, ensuring that only the most pertinent information is passed to the language model.

Our implementation is modular and supports several variants of the RAG pipeline. The core system uses a BM25 retriever to fetch relevant passages for each query, with the option to apply instruction-aware contextual compression to filter and prioritize the most relevant sentences from each retrieved document. We integrate both standard prompt templates and advanced formats, such as the Llama-3.2 user-assistant conversation style and chain-of-thought reasoning prompts, to assess how different prompting strategies affect model performance. For each variant, the retrieved passages are combined with the user’s query into a prompt, which is then tokenized and passed to the language model for answer generation. We carefully control the maximum length of both input and output to prevent truncation and ensure efficient use of the model’s context window.

To evaluate our system, we employ standard metrics for open-domain question answering, including exact match accuracy and ROUGE scores, which provide a comprehensive assessment of both answer correctness and the quality of generated text. We conduct experiments comparing the performance of our custom GPT-small model—trained with rotary positional embeddings, RMSNorm, and grouped query attention—with and without RAG, as well as the large instruction-tuned Llama-3.2-1B-Instruct model in various prompting settings. Our results are averaged across multiple runs to account for generation variability, and detailed experimental configurations—such as learning rates, batch sizes, and token limits—are documented in the appendix. This rigorous experimental setup allows us to systematically evaluate the impact of retrieval, compression, and prompting strategies on the overall effectiveness of our RAG system

4.1 Data

We primarily use the Natural Questions (NQ) dataset for our experiments in open-domain question answering. The NQ dataset consists of real user queries collected from Google, each paired with a relevant Wikipedia article and annotated for both long and short answers. For this project, we focus on the short-answer subset, where the model is expected to generate concise, factual responses based on the retrieved passages. The input to our system is a user query and a set of retrieved passages from the Wikipedia corpus, while the output is the model’s generated answer. This setup closely follows the standard NQ task, ensuring compatibility with established benchmarks.

4.2 Evaluation method

We evaluate model performance using standard metrics for open-domain question answering and summarization tasks. Specifically, we report accuracy (exact match of the predicted answer with the ground truth), as well as ROUGE-1, ROUGE-2, and ROUGE-L scores, which measure the overlap between generated and reference answers at the unigram, bigram, and longest common subsequence levels, respectively. These metrics provide both quantitative and qualitative insights into the quality and correctness of the generated answers. When using chain-of-thought or other advanced prompting strategies, we also perform qualitative analysis of the reasoning steps produced by the model.

4.3 Experimental details

Our experiments are conducted using a custom GPT-small model and a large instruction-tuned LLM (Llama-3.2-1B-Instruct). For retrieval, we use the BM25 algorithm via the Pyserini toolkit to fetch the top-k relevant passages for each query. The retrieved passages are combined with the query into a prompt, which is then fed into the language model for answer generation. We experiment with various prompt formats, including standard prompts and advanced chain-of-thought reasoning templates. For the custom GPT-small model, we use the following configurations: learning rate of $1e4$, batch size of 32, and training for 10 epochs. We employ rotary positional embeddings (RoPE), RMSNorm, and grouped query attention (GQA) for efficient training and inference. For the Llama-3.2-1B-Instruct model, we use the standard Hugging Face Transformers library with default generation parameters unless otherwise specified (e.g., max new tokens of 128, temperature of 0.3). All experiments are run on a single GPU, and detailed hyperparameter settings are provided in the appendix.

4.4 Results

Our experimental results reveal several important patterns that align with established findings in retrieval-augmented generation research. The baseline classification task achieves the highest accuracy (0.59), which is expected given that classification requires only label prediction rather than text generation and synthesis. This performance ceiling demonstrates the inherent complexity introduced when transitioning from discriminative to generative tasks in open-domain question answering.

The core RAG implementations show consistent performance across different configurations. Both RAG Default and LLM + RAG achieve similar accuracy scores (approximately 0.24), indicating that our retrieval pipeline and prompt engineering strategies effectively provide useful contextual information for answer generation. The comparable ROUGE scores (0.18-0.26 across metrics) further suggest that the generated responses maintain reasonable semantic overlap with reference answers, despite the challenging nature of the open-domain QA task.

However, the integration of advanced techniques yields mixed results that highlight important limitations in current RAG methodologies. LLM + RAG + CoT and LLM + RAG + All Techniques both show decreased performance relative to the baseline RAG implementations, with accuracy dropping to 0.237 and 0.201 respectively. This degradation aligns with established research indicating that chain-of-thought prompting requires significantly larger models (typically exceeding 10B parameters) to yield consistent improvements. For smaller instruction-tuned models like our 1B parameter implementation, CoT can introduce verbosity and reasoning artifacts that reduce answer precision rather than enhance it.

The dataset-specific results provide additional insights into retrieval quality and domain adaptation. LLM + RAG + NQ Data shows notably lower performance (0.116 accuracy), suggesting potential overfitting or domain mismatch issues, while LLM + RAG + PRS Data achieves moderate performance (0.210 accuracy). These variations underscore the importance of appropriate dataset selection and domain alignment in RAG systems.

The consistently poor performance of RAG (Older) (0.036 accuracy) and Summary-only configurations confirms that both retrieval quality and proper context integration are critical components for effective RAG implementation. The substantial gap between these baselines and our optimized implementations validates the importance of careful system design and prompt engineering.

A possible reason that ROUGE and accuracy do not improve with advanced prompting or context compression is that these techniques may inadvertently introduce noise or reduce the relevance of the information passed to the model. For example, chain-of-thought prompting can lead to verbose or off-topic reasoning, especially when the underlying language model is not large enough to reliably follow multi-step instructions. Similarly, aggressive context compression may discard useful information, making it harder for the model to generate accurate or complete answers. In both cases, the benefits of reduced input length or more structured reasoning are outweighed by the loss of critical context or the introduction of distracting content, resulting in lower quantitative scores.

These results illustrate a fundamental trade-off between system complexity and performance in RAG architectures. While theoretically beneficial, advanced prompting strategies and additional processing steps can exceed the reasoning capacity of smaller models or introduce context overload that degrades generation quality. This finding suggests that effective RAG implementation requires careful balance

Experiment	accuracy	rouge1	rouge2	rougeL	rougeLsum
Classification (No Generation)	0.5906797663303239				
LLM + RAG + CoT	0.23714504988488105	0.26252249732871413	0.14884030440438706	0.26183990606477214	0.26163428134072864
LLM + RAG + All Techniques	0.20153491941673063	0.21847468170952436	0.1231162467616802	0.21693622868987433	0.21686552255623653
LLM + RAG + NQ Data	0.11619339984650806	0.10771053283716338	0.05855526071949704	0.1068835417588293	0.10699401050897547
LLM + RAG + PRS Data	0.2101304681504221	0.23516558482022765	0.12860784734153866	0.23441994817666378	0.23433997826169728
LLM + RAG	0.2382194934765925	0.18407635427589333	0.10121558794237223	0.18180938231206756	0.18274333196435982
RAG + CoT	0.1355333844973139	0.10730404391379046	0.05415567676120246	0.10613896719806129	0.10626613217940903
RAG Default	0.23960092095165003	0.1856398411003164	0.10110667536607679	0.18337620765863216	0.18424605921919718
RAG (Older)	0.0365310821181888	0.07899620205836602	0.030215071446844273	0.07867878390902155	0.07889892958580574
Summary-only		0.059114703531269755	0.010371038082619823	0.04551631948731147	0.054538251337034716

Figure 1: All Results

between sophistication and model capabilities, with simpler, well-engineered approaches often outperforming complex multi-technique combinations.

5 Analysis

To better understand the strengths and limitations of our system, we conducted a qualitative evaluation by inspecting representative outputs and failure cases across different RAG variants. Our results in rag experiment score comparison show that the baseline classification model achieves the highest accuracy, as expected, since it only requires selecting a label rather than generating or synthesizing text. In contrast, all generation-based RAG variants have lower accuracy and ROUGE scores, reflecting the increased complexity of open-domain question answering, where the model must ground its output in retrieved evidence.

A closer look at the outputs reveals that the core RAG pipeline (RAG Default and LLM + RAG) produces answers that are generally relevant and often factually correct when the retrieval step returns passages closely related to the query. However, errors frequently occur when the retriever surfaces partially relevant or noisy passages, leading the model to generate plausible-sounding but incorrect answers. This highlights the system’s dependence on retrieval quality and the challenge of aligning model generation with external evidence.

Interestingly, advanced prompting techniques such as chain-of-thought (CoT) reasoning and the combination of all techniques do not consistently improve performance over the default RAG setup. In qualitative analysis, we observed that CoT prompts sometimes cause the model to produce verbose or off-topic reasoning steps, especially when the underlying LLM is not sufficiently large or instruction-tuned for multi-step reasoning. This can dilute the relevance of the generated answer and reduce both accuracy and ROUGE scores.

Our experiments with instruction-aware context compression provide further insight into the trade-offs between efficiency and answer quality. By compressing retrieved passages to retain only the most instruction-relevant sentences, we are able to reduce the input length and speed up inference. However, this comes at the cost of lower ROUGE scores and, in some cases, reduced accuracy. Qualitative inspection suggests that aggressive compression can sometimes discard useful context, making it harder for the model to generate fully correct or complete answers. Nevertheless, this trade-off may be acceptable in scenarios where latency or memory constraints are critical, and it demonstrates the practical value of modular, configurable context compression in real-world RAG systems.

Overall, our analysis shows that while our RAG implementation is effective at leveraging external knowledge for open-domain QA, its performance is fundamentally limited by retrieval quality, prompt design, and the capacity of the underlying language model. Simpler, well-engineered approaches often outperform more complex prompting or compression strategies, especially when using mid-sized models. Future improvements could focus on integrating stronger retrievers, more adaptive context compression, and scaling up to larger, more capable LLMs for advanced

6 Conclusion

In this project, we developed and evaluated a modular Retrieval-Augmented Generation (RAG) system using both a custom GPT-style transformer and a large instruction-tuned LLM. Our experiments on the Natural Questions dataset demonstrate several key findings. First, the baseline classification model achieved the highest accuracy, confirming that simple discriminative tasks remain easier than open-domain answer generation. Our RAG pipelines, both with the custom model and Llama-3.2-1B-Instruct, consistently outperformed older baselines and summary-only approaches, showing the effectiveness of retrieval and prompt engineering for grounding answers in external evidence.

However, we observed that advanced prompting strategies—such as chain-of-thought reasoning and multi-technique combinations—did not consistently improve quantitative scores and sometimes even reduced accuracy and ROUGE. This aligns with broader research indicating that such methods often require much larger models to be effective and can introduce verbosity or distract from concise answers in smaller models. Our experiments with instruction-aware context compression further highlighted an important trade-off: while compression reduces input length and speeds up inference, it can also lower ROUGE and accuracy scores by omitting potentially useful information. This suggests that aggressive compression may be best suited for scenarios with strict latency or memory constraints, but should be carefully balanced to avoid discarding relevant context.

Overall, our work confirms that the success of RAG systems depends critically on retrieval quality, prompt design, and the underlying model’s capacity. Simpler, well-tuned approaches often outperform more complex pipelines when using mid-sized models. The primary limitations of our work include reliance on BM25 retrieval (which may miss semantically relevant passages), the limited scale of our models, and the challenge of optimizing context compression without sacrificing answer quality.

For future work, we suggest exploring hybrid or dense retrievers to improve passage relevance, experimenting with adaptive compression strategies that better preserve key information, and scaling up to larger language models for advanced prompting techniques. Additionally, integrating more robust evaluation metrics and human qualitative assessment could provide deeper insights into system behavior and answer faithfulness. This project lays a strong foundation for further research into efficient, reliable, and context-aware retrieval-augmented generation systems.

7 Notes

Google Drive - all of my files are stored here

output/ - Used to save results from experiments, such as model predictions, evaluation outputs, generated texts, and final reports.

cache/ - Stores cached files generated during data processing or model training. This can include preprocessed datasets, intermediate results, or Hugging Face dataset/model caches to speed up repeated runs.

logs/ - Holds log files generated during training, evaluation, or inference.

]

References

- [1] H. Hou, F. Ma, B. Bai, X. Zhu, and F. Yu, "Enhancing and Accelerating Large Language Models via Instruction-Aware Contextual Compression," arXiv preprint arXiv:2408.15491, 2024.
- [2] H. Touvron, T. Lavril, G. Izacard, et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.
- [3] T. B. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

- [5] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [6] Y. Hou, Z. Zhang, et al., "Enhancing and Accelerating Large Language Models via Instruction-Aware Contextual Compression," arXiv preprint arXiv:2305.12754, 2023.

]